```cpp
#pragma once
#include <iostream>
#include <functional>
#include <cmath>
#include "matrix.hpp"

using namespace std;
inline double sigmoide(double x){
    return (1 / (1.0 + exp(-x)));
}
inline double derivadaSigmoide(double a){
    return a * (1.0 - a);
}

inline double ReLU(double x){
    if(x > 0){
        return x;
    }else{
        return 0;
    }
}
inline double derivadaReLU(double x){
    if(x > 0){
        return 1;
    }else if(x <= 0){
        return 0;
    }
}

inline double tangenteHiperbolica(double x){
    return tanh(x);
}
inline double derivadaTanHiper(double x){
    return 1.0 - x*x;
}




class capaDensa {
private:
    bool esSoftmax_ = false;
    Matriz W, B, X, Z, A;
    function<double(double)> activacion_, derivadaActivacion_;

    // Acumuladores de Adam
    Matriz mW, vW; // Momentos para pesos (W)
    Matriz mB, vB; // Momentos para sesgos (B)

    // Hiperparámetros de Adam
    double beta1 = 0.9;
    double beta2 = 0.999;
    double epsilon = 1e-8;

public:
    capaDensa(size_t entradas, size_t salidas,
              function<double(double)> act = sigmoide,
              function<double(double)> dAct = derivadaSigmoide) {

        activacion_ = act;
        derivadaActivacion_ = dAct;

        W = Matriz(salidas, entradas);
        B = Matriz(salidas, 1);
```

```
        // Inicializar acumuladores en ceros
        mW = Matriz(salidas, entradas, 0.0);
        vW = Matriz(salidas, entradas, 0.0);
        mB = Matriz(salidas, 1, 0.0);
        vB = Matriz(salidas, 1, 0.0);

        // Inicialización Xavier/He
        double limite = sqrt(6.0 / (entradas + salidas));
        W.aleatorizar(-limite, limite);
        B.aleatorizar(0.0, 0.0);
    }

    ~capaDensa() {}
    void usarSoftmax() {
        esSoftmax_ = true;
    }

    Matriz forward(const Matriz& X_in) {
        this->X = X_in;
        Z = W.matmul(X) + B;

        // El Bypass
        if (esSoftmax_) {
            A = Z.softmax();
        } else {
            A = Z.aplicar(activacion_);
        }
        return A;
    }



    // Se agrega el parámetro 't' (timestep) requerido por Adam
    Matriz backward(const Matriz& gradienteSalida, double tasaDeAprendizaje, int t) {
        Matriz Delta;

        if (esSoftmax_) {
            // ATAJO MATEMÁTICO: Si es Softmax + CrossEntropy, el gradiente ya viene
resuelto
            Delta = gradienteSalida;
        } else {
            // Comportamiento normal (derivada de la activación)
            Matriz dA = A.aplicar(derivadaActivacion_);
            Delta = gradienteSalida.hadamard(dA);
        }

        // ... (El resto del código de backward con Adam se queda exactamente igual)
        Matriz dW = Delta.matmul(X.transpuesta());
        Matriz dX = W.transpuesta().matmul(Delta);

        // 1. Actualización Adam para Pesos (W)
        for (size_t i = 0; i < W.filas(); ++i) {
            for (size_t j = 0; j < W.columnas(); ++j) {
                double g = dW(i, j);

                // Acumular momentos
                mW(i, j) = beta1 * mW(i, j) + (1.0 - beta1) * g;
                vW(i, j) = beta2 * vW(i, j) + (1.0 - beta2) * (g * g);

                // Corrección de sesgo
                double m_hat = mW(i, j) / (1.0 - pow(beta1, t));
                double v_hat = vW(i, j) / (1.0 - pow(beta2, t));

                // Actualizar peso
                W(i, j) = W(i, j) - (tasaDeAprendizaje * m_hat) / (sqrt(v_hat) + epsilon
);
```

```
            }
        }

        // 2. Actualización Adam para Sesgos (B)
        for (size_t i = 0; i < B.filas(); ++i) {
            double g = Delta(i, 0);

            // Acumular momentos
            mB(i, 0) = beta1 * mB(i, 0) + (1.0 - beta1) * g;
            vB(i, 0) = beta2 * vB(i, 0) + (1.0 - beta2) * (g * g);

            // Corrección de sesgo
            double m_hat = mB(i, 0) / (1.0 - pow(beta1, t));
            double v_hat = vB(i, 0) / (1.0 - pow(beta2, t));

            // Actualizar sesgo
            B(i, 0) = B(i, 0) - (tasaDeAprendizaje * m_hat) / (sqrt(v_hat) + epsilon);
        }

        return dX;
    }

    void guardar(ofstream& out) const {
        W.guardarArchivo(out);
        B.guardarArchivo(out);
    }

    void cargar(ifstream& in) {
        W.cargarArchivo(in);
        B.cargarArchivo(in);
    }
    void guardarBinario(std::ofstream& out) const {
        W.guardarBinario(out);
        B.guardarBinario(out);
    }
    void cargarBinario(std::ifstream& in) {
        W.cargarBinario(in);
        B.cargarBinario(in);
    }
};
```